Volume 01 - Startup, Flask Registration, and Page Routes

Source revision: `4405d167fc323f33be76e2c7f6614fd21cc574a8` **Files:** `run.py`, `app/__init__.py`, and `app/routes.py` lines 1-233. **Code excerpt policy:** the PDF builder inserts the full, consecutive source ranges from this exact checkout, with original line numbers. The teaching notes below explain these blocks; no source code is silently rewritten.

1. What runs first?

`run.py` - local entry point (lines 1-6)

The file imports `os` and the package-level `app`. When executed as a script, it reads `PORT` (default 5000) and `FLASK_DEBUG` (default false), then calls Flask's development server with host `0.0.0.0`. The `if __name__ == '__main__'` guard means this body runs for `python run.py`, but not when Gunicorn imports the module.

• `os.environ.get('PORT', 5000)` returns a string environment value if present, so `int(...)` converts it for the server argument. A non-numeric `PORT` raises `ValueError` before the server starts.

• Debug is enabled only for exact strings `'1'`, `'true'`, or `'True'`. Values such as `'TRUE'` are false.

• `0.0.0.0` listens on all container interfaces. It is needed for a hosted process to be reachable through the platform's network proxy; it is not the public URL.

• **Verified distinction:** local `python run.py` enters the `app.run(...)` code. The `Procfile` instead asks Gunicorn to import `run:app`; the `__main__` body does not run in that import path.

`app/__init__.py` - app construction and route registration (lines 1-5)

`app = Flask(__name__)` creates one Flask instance. `__name__` tells Flask which package/module roots to use for resources. The subsequent `from app import routes` executes `routes.py`; importing `app` has already assigned the `app` object, so the routes module can import that same object and apply `@app.route(...)` decorators. Importing routes is the registration side effect: Flask records each URL/method and its handler on the app.

This is not an application factory: the module creates a single global app at import time. That is simple for this small service, but makes startup tightly coupled to files read by imported modules and complicates tests that need isolated app configurations.

Gunicorn path

`Procfile` contains `web: gunicorn run:app`. Gunicorn imports module `run`, finds the global `app`, then serves requests using its WSGI interface. `run.py` imports `app`, triggering the same module initialization and route registration. It does not call `app.run()` in this mode.

2. Route-module imports and module-level initialization

`app/routes.py` lines 1-47

This module is both HTTP layer and composition root for one application process. Its imports fall into groups:

• Standard library: `os`, `json`, `datetime/date`, `ZoneInfo`.

• Flask: `render_template`, `request`, `jsonify`.

• Data table loading: pandas.

• Application modules: Flask app, topology graph functions, forecasting functions, schedule optimizer, calendar context, service-window predicate.

At import time it computes `BASE_DIR` by taking the absolute current source-file path and moving up two directories. `STATIONS_CLEANED_PATH` points to `data/cleaned/stations_cleaned.csv`; if that path does not exist, it switches to legacy `csvFiles/DELHI_METRO_DATA.csv`. It then reads the selected CSV into `stations_df`, strips source marker characters from station names, creates a sorted unique station-name list, and builds `travel_graph` once.

Important consequences:

1. CSV parsing and graph construction occur during import/startup, not inside every page route. 2. A missing required CSV, malformed schema, or error in graph building may stop process startup. 3. The fallback only checks whether the cleaned file exists. It does not prove the legacy CSV has every column the rest of the module expects. 4. `stations_df` is a module-global DataFrame shared by all requests in this process. The handlers read it but do not intentionally mutate it after initialization. 5.

`BUSIEST_STATION = "Botanical Garden"` and `PEAK_HOUR = "09:00"` are fixed display/example defaults. They are not dynamically computed network-wide benchmarks in this module.

Dependency chain / circular-import question

`app/__init__.py` defines `app` before importing `routes`. `routes.py` then executes `from app import app`, which retrieves the already-defined symbol from the partially initialized package. This import order works as written. If someone moved the routes import above app construction, the expected symbol would not yet exist.

3. `_parse_datetime(date_str, time_str)`

Signature: `_parse_datetime(date_str, time_str)` **Input:** date/time strings or false-like empty values. Types are not annotated. **Output:** `(dt_obj, hour, minute)`, where `dt_obj` is a naive `datetime` and hour/minute are integers. **Raises:** `ValueError` with user-facing text on malformed nonempty date or time.

Execution order:

1. Capture today's date using `datetime.now(ZoneInfo('Asia/Kolkata')).date()`. The clock is read in Delhi's time zone, avoiding the host's local zone for the default date. 2. If the date string is empty, use today. Otherwise parse exactly `%Y-%m-%d`. `datetime.strptime` rejects impossible dates such as `2026-02-30`; the `except ValueError` replaces its implementation-specific message with a stable prompt. 3. If the time string is empty, choose 09:00. Otherwise parse `%H:%M`; it expects a 24-hour hour-and-minute representation and rejects malformed values. 4. Construct `datetime(year, month, day, hour, minute)`. This object is naive: it contains no timezone information even though the default date used an aware clock. Downstream code treats its date/hour as local Delhi scenario fields. 5. Return the datetime and its already-parsed hour/minute separately for convenience.

Illustrative trace: date `2026-10-14`, time `09:15` produces `datetime(2026, 10, 14, 9, 15)`, `9`, `15`. Empty date/time uses the current Kolkata/Delhi calendar date and 09:00. A valid but closed time such as `02:02` is parsed normally; closure is a separate service-window decision.

Risks: no future-date restriction is enforced here; no station validation is done here; values of the wrong type may fail before the caught `ValueError` (for example, `strptime` expects text). The API handler's broadness around malformed JSON types is discussed in Volume 02.

4. `_hour_band(profile, hour, minute=0)`

It labels selected time as closed, unavailable, peak, lower or typical based on a station/date's predicted hourly profile.

1. Call `is_service_operating(hour, minute)` first. A closed time returns before profile indexing. 2. Sort the profile values into a new list. Empty profile returns `Unavailable`. 3. Select low/high values at integer positions `int((n-1)*.25)` and `int((n-1)*.75)`. This is an index-based quartile approximation, not an interpolated statistical percentile. 4. Index `profile[hour - 5]`, relying on a profile whose first item corresponds to hour 05. If hour is outside 05-23 while service is marked open due a bad service rule/input, indexing could fail; with the current window it does not happen for valid values. 5. Values at/above `high` are Peak demand; values at/below `low` are Lower demand; the rest are Typical demand. If all values are equal, `high == low`, so the first condition wins and every value is classified Peak.

Complexity is O(n log n) time because sorting dominates, O(n) additional list space. A selection algorithm could find quartile order statistics without sorting all values, but n=19 makes this negligible.

5. `landing()` - GET `/`

The decorator registers a GET endpoint. The handler obtains today's Delhi date, fixes the sample station to Rajiv Chowk, computes a full-day scenario at 09:00, and passes labels/profile/metrics to `landing.html`. The template can render a sample visualization without a user request. The resulting profile is generated by the same inference helper used by the dashboard; it is a model estimate on synthetic-trained data, not observed ridership.

Inputs: no route parameters. **Output:** rendered HTML response. **Calls:** `predict_station_full_day_profile`, `get_model_evaluation_summary`, `render_template`. A startup/model error would already have happened when imports loaded artifacts.

6. `index()` - GET `/dashboard`

This handler converts the module-level pandas table into `stations_meta`, a dictionary keyed by station name. For each DataFrame row:

• `iterrows()` yields an index and a Series; _ discards the index.

• It reads canonical station name and uses `row.get('All_Lines', row['Line'])` as a fallback. The value is converted to string, comma-split, and whitespace-trimmed into a list.

• It builds metadata: primary line, all lines, interchange boolean (using explicit column if available, else infer from multiple lines), and coordinates converted to floats only if pandas says they are not missing.

It then gets stored evaluation summary and renders `index.html` with station list, metadata, constants, current Delhi date, fixed default time 09:00 and Rajiv Chowk station. `default_date` and `today_date` call the clock separately, so at midnight they could theoretically disagree by one day; a single captured date would be more consistent.

Output contract: template receives `stations`, `stations_meta`, `busiest_station`, `peak_hour`, `eval_metrics`, `default_date`, `today_date`, `default_time`, `default_station`. In `index.html`, `stations_meta` is serialized to `window.STATIONS_META`, which `dashboard.js` uses to filter line choices.

Complexity: O(S) iteration plus sorting O(S log S) at module initialization for `ALL_STATIONS`; each dashboard request rebuilds a dictionary for S station rows. With 228 rows this is small, though metadata could be built once.

7. `get_footfall()` - legacy POST `/get_footfall`

This synchronous result route accepts an HTML form. It reads station/day/time/date/line from `request.form`, strips station marker characters, parses the datetime, derives weekday/service/calendar context, and then calls forecast and schedule functions. It computes the full daily profile even when the selected time is closed; selected-time and hourly demand are set to zero when closed. It calls the station optimizer with explicit `service_open` and chooses the selected line's recommendation for headline fields if a matching one exists. If line doesn't match, the primary recommendation remains selected.

Travel estimates come from `shortest_times_from(travel_graph, station)`, are associated with the station's first line from `stations_df`, and are sorted by minutes. Here the travel-time block has a broad `except Exception` that converts any error into an empty list. This preserves page rendering but hides the exception cause unless separately logged; contrast with `/api/predict`, whose graph call is not inside this catch.

Finally `render_template('result.html', ...)` passes numerous flattened headline values plus nested `schedule_res`, `profile_24h`, travel info, and provenance metrics. The template is the consumer contract; mismatched key names can fail rendering or display blanks.

Important implementation boundary: this route is not the asynchronous dashboard API. It produces a complete page response after form submit. The modern dashboard uses `/api/predict` and browser-side rendering. `day_str` is read but never used; actual weekday is calculated from parsed date. This is dead/legacy form input.

8. Source-to-flow map for Volume 01

Caller / trigger	Function	Calls / data	Next stage
`python run.py`	`run.py` main guard	imports global `app`, reads env	Flask development server
Gunicorn `run:app`	module import	`app/__init__.py` -> routes import	WSGI request handling
`GET /`	`landing()`	profile inference, evaluation summary	Jinja `landing.html`
`GET /dashboard`	`index()`	global station DataFrame, metrics	Jinja `index.html`, then JS
`POST /get_footfall`	`get_footfall()`	parser, calendar, forecast, schedule, graph	Jinja `result.html`

9. Interview questions and answers

Why does importing `app` register routes? `app/__init__.py` imports `routes`; evaluating each decorated function applies its `@app.route` decorator, registering the handler.

Why is the graph built once? Topology and station metadata are static during a process, so building once avoids repeated CSV reads/graph creation. Trade-off: changes to files require process restart; startup failures happen earlier.

Is `dt_obj` timezone-aware? No. The current Delhi date default is obtained from an aware `ZoneInfo` clock, but the constructed scenario `datetime` has no `tzinfo`.

Does `_hour_band` compute a standard percentile? No. It sorts and selects two discrete array positions. For small profile length 19, this is a simple quartile-style threshold.

How would you test parser boundaries? Check empty defaults, valid leap date, impossible date, malformed format, `00:00`, `23:59`, and wrong-type values at API boundary. Keep parser behavior separate from whether the time is an operating hour.

10. Modification exercise and solution

Exercise: Make `index()` read the Delhi date once so `default_date` and `today_date` cannot differ at midnight.

Solution: At the start of the handler, set `today = datetime.now(ZoneInfo('Asia/Kolkata')).date().isoformat()`, then pass `today` for both template fields. This preserves the existing template contract while removing two clock reads. Add a focused test with a patched clock that would cross midnight between calls; no such test currently exists in the repository.

Current coverage status

This lesson covers startup and synchronous route code. It does not yet explain `/api/predict`; that request orchestration receives its own source-ordered lesson because it is the central runtime path.

Appendix A - complete source excerpts

These excerpts come from the documented commit. Original line numbers are printed at left. For print readability, Unicode dash variants are normalized to ASCII hyphens and other non-ASCII source symbols are shown as explicit Unicode escape notation; this does not change the source line numbers or code meaning. Long lines wrap with indented continuations.

Local entry point `run.py:1-7`

```
1  import os
2  from app import app
3
4  if __name__ == '__main__':
5      port = int(os.environ.get('PORT', 5000))
6      debug = os.environ.get('FLASK_DEBUG', '0') in ('1', 'true', 'True')
7      app.run(host='0.0.0.0', port=port, debug=debug)
```

Flask construction and route import `app/__init__.py:1-5`

```
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  from app import routes
```

Route-module imports and module initialization (lines 1-47) `app/routes.py:1-47`

```
1  """
2  DMRC Passenger Footfall Prediction & Train Scheduling Optimization Web Application.
3
4  Routes:
5  - /                  : Interactive transit management dashboard
6  - /get_footfall      : Synchronous result view (backward-compatible)
7  - /api/predict       : JSON endpoint for real-time forecasting & scheduling optimization
8  - /api/stations      : Station directory with line and interchange metadata
9  - /api/model_metrics : Model validation metrics vs historical baselines
10 """
11 import os
12 import json
13 from datetime import datetime, date
14 from zoneinfo import ZoneInfo
15 from flask import render_template, request, jsonify
16 import pandas as pd
17
18 from app import app
19 from app.travel_graph import build_graph, shortest_times_from
20 from app.forecasting import (
21     get_station_info,
22     predict_hourly_footfall,
23     predict_footfall_with_minute_interpolation,
24     predict_station_full_day_profile,
25     get_model_evaluation_summary
26 )
27 from app.scheduling_optimizer import optimize_station_schedule
28 from app.calendar_context import get_calendar_context
29 from app.operating_hours import is_service_operating
30
31 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
32 STATIONS_CLEANED_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'stations_cleaned.csv')
33 if not os.path.exists(STATIONS_CLEANED_PATH):
34     STATIONS_CLEANED_PATH = os.path.join(BASE_DIR, 'csvFiles', 'DELHI_METRO_DATA.csv')
35
36 # Load cleaned stations metadata
37 stations_df = pd.read_csv(STATIONS_CLEANED_PATH)
38 stations_df['Station'] = stations_df['Station'].astype(str).str.rstrip('†‡*#¤').str.strip()
39 ALL_STATIONS = sorted(stations_df['Station'].unique().tolist())
40
41 # Build line-topology graph once at startup
42 travel_graph = build_graph(STATIONS_CLEANED_PATH)
43
44 # Network-wide apex benchmarks
45 BUSIEST_STATION = "Botanical Garden"
46 PEAK_HOUR = "09:00"
47
```

Date/time parser `app/routes.py:49-72`

```
def _parse_datetime(date_str, time_str):
    """
    Safely parses date and time inputs into datetime object, hour, and minute.
    """
    today = datetime.now(ZoneInfo('Asia/Kolkata')).date()
    if not date_str:
        dt_date = today
    else:
        try:
            dt_date = datetime.strptime(date_str, "%Y-%m-%d").date()
        except ValueError:
            raise ValueError("Choose a valid date in YYYY-MM-DD format.")

    if not time_str:
        hour, minute = 9, 0
    else:
        try:
            parsed_time = datetime.strptime(time_str, "%H:%M").time()
            hour, minute = parsed_time.hour, parsed_time.minute
        except ValueError:
            raise ValueError("Choose a valid time in HH:MM format.")

    dt_obj = datetime(dt_date.year, dt_date.month, dt_date.day, hour, minute)
    return dt_obj, hour, minute
```

Demand band classifier `app/routes.py:75-89`

```
def _hour_band(profile, hour, minute=0):
    """Classify an hour relative to this station/date's predicted profile."""
    if not is_service_operating(hour, minute):
        return "Metro closed"
    values = sorted(profile)
    if not values:
        return "Unavailable"
    low = values[int((len(values) - 1) * 0.25)]
    high = values[int((len(values) - 1) * 0.75)]
    value = profile[hour - 5]
    if value >= high:
        return "Peak demand"
    if value <= low:
        return "Lower demand"
    return "Typical demand"
```

Landing page route `app/routes.py:92-108`

```
@app.route('/')
def landing():
    """Product landing page with a live, clearly labeled scenario preview."""
    today = datetime.now(ZoneInfo('Asia/Kolkata')).date()
    hero_station = 'Rajiv Chowk'
    hero_profile = predict_station_full_day_profile(
        hero_station,
        datetime(today.year, today.month, today.day, 9, 0)
    )
    return render_template(
        'landing.html',
        default_date=today.isoformat(),
        default_date_label=today.strftime('%A, %d %B %Y'),
        hero_station=hero_station,
        hero_profile=hero_profile,
        eval_metrics=get_model_evaluation_summary(),
    )
```

Dashboard route `app/routes.py:111-139`

```
@app.route('/dashboard')
def index():
    # Build stations metadata dict for client-side search and line filtering
    stations_meta = {}
    for _, row in stations_df.iterrows():
        st = row['Station']
        all_lines = [l.strip() for l in str(row.get('All_Lines', row['Line'])).split(',')]
        stations_meta[st] = {
            "line": row['Line'],
            "all_lines": all_lines,
            "is_interchange": bool(row.get('Is_Interchange', len(all_lines) > 1)),
            "latitude": float(row['Latitude']) if pd.notna(row['Latitude']) else None,
            "longitude": float(row['Longitude']) if pd.notna(row['Longitude']) else None,
        }

    eval_metrics = get_model_evaluation_summary()

    return render_template(
        'index.html',
        stations=ALL_STATIONS,
        stations_meta=stations_meta,
        busiest_station=BUSIEST_STATION,
        peak_hour=PEAK_HOUR,
        eval_metrics=eval_metrics,
        default_date=datetime.now(ZoneInfo('Asia/Kolkata')).date().isoformat(),
        today_date=datetime.now(ZoneInfo('Asia/Kolkata')).date().isoformat(),
        default_time="09:00",
        default_station="Rajiv Chowk"
    )
```

Legacy synchronous result route `app/routes.py:142-231`

```
142 @app.route('/get_footfall', methods=['POST'])
143 def get_footfall():
144     station = request.form.get('station', 'Rajiv Chowk').strip().rstrip('†‡*#¤').strip()
145     day_str = request.form.get('day', 'Monday')
146     time_str = request.form.get('time', '09:00')
147     date_str = request.form.get('date', '')
148     selected_line = request.form.get('line', '')
149 
150     try:
151         dt_obj, hour, minute = _parse_datetime(date_str, time_str)
152     except ValueError as exc:
153         return str(exc), 400
154     day_name = dt_obj.strftime('%A')
155     formatted_time = f"{hour:02d}:{minute:02d}"
156     service_open = is_service_operating(hour, minute)
157     calendar_context = get_calendar_context(dt_obj.date())
158     calendar_context['service_open'] = service_open
159 
160     st_info = get_station_info(station)
161 
162     # 1. Machine learning footfall forecast
163     time_footfall = predict_footfall_with_minute_interpolation(station, dt_obj, hour, minute) if service_open
    else 0
164     hourly_footfall = predict_hourly_footfall(station, dt_obj, hour) if service_open else 0
165     profile_24h = predict_station_full_day_profile(station, dt_obj)
166     total_footfall = profile_24h['daily_total']
167 
168     # 2. Scheduling optimization
169     schedule_res = optimize_station_schedule(
170         station=station,
171         primary_line=st_info['line'],
172         all_lines=st_info['all_lines'],
173         hour=hour,
174         day_of_week=dt_obj.weekday(),
175         total_footfall=hourly_footfall,
176         is_workday=calendar_context['is_workday'],
177         service_open=service_open
178     )
179 
180     # If user selected a specific line, prioritize its recommendation for the headline cards
181     primary_rec = schedule_res['primary_recommendation']
182     if selected_line:
183         for rec in schedule_res['line_recommendations']:
184             if rec['line'].lower() == selected_line.lower():
185                 primary_rec = rec
186                 break
187 
188     # 3. Network travel times from topology to ALL stations
189     try:
190         times = shortest_times_from(travel_graph, station)
191         station_line_map = dict(zip(stations_df['Station'], stations_df['Line']))
192         travel_info = [
193             {
194                 'End Station': dest,
195                 'Travel Time (minutes)': round(mins, 1),
196                 'Line': station_line_map.get(dest, 'Metro Network')
197             }
198             for dest, mins in sorted(times.items(), key=lambda kv: kv[1])
199         ]
200     except Exception:
201         travel_info = []
202 
203     return render_template(
204         'result.html',
205         station=station,
206         station_info=st_info,
207         day=day_name,
208         calendar_context=calendar_context,
209         selected_hour_band=_hour_band(profile_24h['predicted'], hour, minute),
210         date=dt_obj.strftime('%Y-%m-%d'),
211         time=formatted_time,
212         total_footfall=total_footfall,
213         time_footfall=time_footfall,
214         hourly_footfall=hourly_footfall,
215         num_trains_needed=round(primary_rec['recommended_trains_per_hour'], 1),
216         interval=primary_rec['recommended_headway_min'],
217         existing_interval=primary_rec['existing_headway_min'],
218         existing_trains=round(primary_rec['existing_trains_per_hour'], 1),
219         capacity_utilization=primary_rec['capacity_utilization_pct'],
220         crowding_badge=primary_rec['crowding_badge'],
221         crowding_color=primary_rec['crowding_color'],
222         avg_wait_time=primary_rec['avg_wait_time_rec_min'],
223         service_open=service_open,
224         explanation=primary_rec['explanation'],
225         schedule_res=schedule_res,
226         profile_24h=profile_24h,
227         busiest_station=BUSIEST_STATION,
228         peak_hour=PEAK_HOUR,
229         travel_info=travel_info,
230         eval_metrics=get_model_evaluation_summary()
231     )
```